declarators or identifiers. The declaration begins with optional storage class specifiers, type specifiers, and other modifiers. The identifiers are separated by commas and the list is terminated by a semicolon.

Declarations of variable identifiers have the following pattern:

```
storage-class [type-qualifier] typevar1 [=init1], var2
[=init2], ... ;
```

where var1, var2,... are any sequence of distinct identifiers with optional initializers. Each of the variables is declared to be of type; if omitted, type defaults to int. Specifierstorage-class can take values extern, static, register, or the default auto. Optional type-qualifier can take values constant or volatile.

For example:

```
/* Create 3 integer variables called x, y, and z
     and initialize x and y to the values 1 and 2,
respectively: */
int x = 1, y = 2, z;    // z remains uninitialized

/* Create a floating-point variable q with static
modifier,
    and initialize it to 0.25: */
static float q = .25;
```

These are all defining declarations; storage is allocated and any optional initializers are applied.

4.2.8 Functions

Functions are central to C programming. Functions are usually defined as subprograms which return a value based on a number of input parameters. Return value of a function can be used in expressions – technically, function call is considered to be an expression like any other.

C allows a function to create results other than its return value, referred to as side effects. Often, function return value is not used at all, depending on the side effects. These functions are equivalent to procedures of other programming languages, such as Pascal. C does not distinguish between